

ML Intern Assignment

Insurance Enrollment Prediction + Outreach Assistant Agent

Objective

Build an end-to-end machine learning system that:

1. Predicts whether an employee will opt in to a new voluntary insurance product, using a multi-file, deliberately imperfect HR/benefits dataset that is not the plain "predict insurance enrollment" tutorial dataset, and
2. Exposes that model through a small tool-using Outreach Assistant agent that ranks employees for a benefits team with a limited outreach budget, and can explain individual predictions without leaning on protected attributes.

The assignment is designed to assess:

- Your ability to investigate multi-table HR data, join tables correctly, and engineer features under real messiness (mixed date formats, sentinel codes, inconsistent categorical labels, conflicting duplicate records).
- Your judgment about which fields are legitimate predictors versus target leakage, and which fields are legitimate predictors but ethically/legally risky to act on in an insurance context.
- Your ability to wrap a model in a simple agentic interface (tools + ranking + refusal rules), not just a notebook metric.

Python is expected. A normal laptop is enough, no GPU required.

Attachments: the two CSV files described under Data are attached to the email that sent you this document. We are not sharing any starter code or repository, you build the full solution from scratch.

Core Functional Requirements

1. Multi-Table Data Investigation & Cleaning

You receive two files (see Data). You must join them thoughtfully.

At minimum:

- Profile both tables: missingness, dtypes, cardinality, duplicates, impossible values.
- Resolve or explicitly policy-handle duplicate employee_id rows with conflicting enrolled labels pick a policy (drop both, keep most recent contact, majority vote across the pair, etc.) and state it.
- Parse the messy operational fields: application_date (mixed formats / blank), last_contact_date (some rows show contact happening after the application, flag or fix, don't silently feed it to a model), inconsistent casing in last_contact_channel (Email / EMAIL / e-mail / ...), and free-text plan_tier_requested (Premium / premium plan / PREMIUM / Gold Plan / ...).
- Handle the prior_year_enrolled sentinel: 1 means "no prior-year record (new hire)", not "not enrolled." Treating it as a literal 0/1 numeric feature without addressing the sentinel is a real bug, not a stylistic nitpick.
- Document every non-trivial cleaning decision.

2. Feature Engineering

- Engineer features from the messy dates, contact-channel, and plan-tier text.
 - Join the region table and reason about which region-level columns would realistically be available at the moment you'd need to make a prediction.
 - Classify every feature as usable, analysis-only, or forbidden / leaky, and justify the split.
 - At least one field nearly reconstructs the target outright treat any suspiciously strong predictor with scrutiny, not blind trust.
 - Fairness / compliance checkpoint (required): some demographic fields are present and mildly predictive. Using them to prioritize who gets proactive outreach can create disparate treatment even unintentionally. State whether your final model uses them and why; if it does, report a subgroup performance breakdown and comment on whether any group looks disadvantaged. There's no single correct answer we grade the reasoning, not the choice.

3. Model Training & Honest Evaluation

- Train a classifier for enrolled.
- Proper train/test split (or cross-validation), this is cross-sectional data, not a time series, so a plain stratified split is fine, but say so.
- Report AUC-ROC and a business-relevant ranking metric (e.g. Precision@K or Lift@K, see the outreach framing in §4).
- Beat at least one naive baseline (majority class, or a simple rule like "predict 1 if has_dependents == Yes").
- Report a second evaluation with the leaky column removed if you experimented with it, graders will check.

4. Agentic Layer, Outreach Assistant (Required)

The benefits team cannot personally contact all 10,000 employees every enrollment window, each region has a fixed `hr_outreach_capacity` in the region table. Build a small tool-using agent (LLM optional) that helps the benefits team decide who to contact and can explain why, without hard-coding answers.

Minimum tool capabilities (names may differ):

Capability	Purpose
<code>predict_enrollment</code>	Run your trained model on cleaned features for one employee
<code>rank_outreach_candidates</code>	Given a region and its <code>hr_outreach_capacity</code> , return the top-K prioritized employee list
<code>lookup_region_profile</code>	Return region-level stats from <code>region_benefit_profiles.csv</code>
<code>explain_prediction</code> (recommended)	Generate a short natural-language explanation for one prediction

Agent behaviour requirements:

- Must support at least 5 demo queries (notebook or script output is fine), e.g. "who are the top 20 outreach priorities in the Midwest this window?", "why is employee X predicted to enroll?", "what's wrong with this raw row?".

- Must refuse to use `legacy_propensity_score` (or any other field you identified as reconstructing the target) when asked for a prediction or explanation. Refusal should be explicit in the response.
- `explain_prediction` must not justify a prediction by citing gender, marital_status, or age in the generated text (e.g. it must not say "...because they are Male and Married"), even if those fields were used inside the model. This is a deliberate test of whether you think about how a generated explanation can launder a decision that would look bad, or be legally risky, if said out loud to an employee or regulator. If your final model doesn't use those fields at all, say so and this requirement is automatically satisfied, that's a legitimate design choice.
- If you use an LLM API, document the model + prompts. If you use a pure-Python router/template (no LLM), that is fully acceptable, tool boundaries and the refusal behavior matter more than the framework.
- We are not providing starter code for the agent, design and build the tool router yourself as part of your submission.

5. Integrity of Reasoning

- Every cleaning / leakage / fairness / agent-refusal decision must be explainable in a live walkthrough.
- AI coding assistants are allowed. Declare them in `AI_USAGE.md`.
- Copying a generic "predict insurance enrollment" notebook without adapting to this schema (region table, sentinels, leak column, agent, fairness checkpoint) will not pass.

Data

We are not providing a starter repository or any code, this email's two attachments are the only files you receive from us:

- `employees_raw.csv`
- `region_benefit_profiles.csv`

Save both into a `data/` folder in your own project. Everything else (cleaning, feature engineering, modelling, agent, docs) is built by you from scratch.

`employees_raw.csv`, one row per employee (~10k rows, includes 8 planted duplicate-id conflicts)

Column	Meaning
<code>employee_id</code>	Employee identifier (not guaranteed unique, check)
<code>age</code> , <code>gender</code> , <code>marital_status</code>	Demographics, see the fairness checkpoint in §2
<code>salary</code>	Annual salary (a few values are implausibly low, investigate)
<code>employment_type</code>	Full-time / Part-time / Contract
<code>region</code>	Join key to <code>region_benefit_profiles.csv</code>
<code>has_dependents</code>	Yes/No
<code>tenure_years</code>	Years at company (a few values are inconsistent with age, investigate)
<code>application_date</code>	When the employee responded to the enrollment offer (mixed formats / blank)
<code>last_contact_date</code>	When benefits ops last contacted them (a few rows are chronologically impossible vs <code>application_date</code>)
<code>last_contact_channel</code>	Email / Phone / SMS / None, inconsistent casing/spelling on purpose
<code>plan_tier_requested</code>	Free-text plan tier, inconsistent casing/naming on purpose

broker_channel	How the plan was sourced (some blank)
prior_year_enrolled	0/1, or, 1 meaning "no prior-year record" (new hire)
legacy_propensity_score	A score from the old rules-based outreach system, captured at contact time, validate before treating as an innocent feature
outreach_notes	Free-text notes (mostly harmless; read before blindly vectorizing)
enrolled	Target, 1 if enrolled, 0 if not

region_benefit_profiles.csv, one row per region

Column	Meaning
region	Join key to employees
n_employees_region	Employee count in this scrape
hist_enrollment_rate_region	Enrollment rate computed from this same scrape, think about whether this is safe to use as a feature without out-of-fold discipline
avg_salary_region	Average salary in this scrape
avg_premium_cost_usd	Average premium cost for the region
benefits_broker_rating	0-5 broker quality rating
hr_outreach_capacity	How many employees benefits ops can personally contact this window, this is your top-K budget for the agent's ranking tool
open_enrollment_window_days	Length of the current enrollment window
state_mandate_level	Ordinal risk/mandate level, labels are dirty on purpose ("low", "MED", ...)

We have not cleaned these files. Finding what is wrong is part of the exercise.

Extended (Optional) Scope

- Probability calibration (Brier score / reliability curve), relevant because the agent's ranking tool consumes raw probabilities, not just a 0/1 label.
- Out-of-fold region aggregate encoding for hist_enrollment_rate_region.
- LLM-backed agent with streaming explanations (must still enforce both refusal rules above).
- A tiny CLI or Streamlit front-end for the assistant.

Technology Expectations

Suggested Stack

- Python 3.10+, pandas, scikit-learn (or XGBoost / LightGBM)
- Agent: pure Python tools, or LangChain / OpenAI tool calling / Anthropic tools, your choice
- Optional: matplotlib / plotly

Architecture

Separate clearly:

- Cleaning / joins

- Feature engineering
- Training / evaluation
- Agent tools + demo queries

Deliverables

1. GitHub repository

- Runnable training + evaluation code
- Agent code (built from scratch, no starter file was provided) + demo of ≥ 5 queries
- requirements.txt, run instructions
- Include the two attached CSVs in the repo (or .gitignore them and say so clearly in the README if you prefer to keep the repo small, either is fine)

2. report.md (2–4 pages)

- Data issues found (including the two "unplanted" base-file issues + the new operational fields + join issues)
- Feature choices, especially what you refused to use and why, and how you handled the fairness checkpoint
- Model + baselines + subgroup performance breakdown
- Agent design: tools, refusal rules, example transcripts
- What you'd do with more time

3. AI_USAGE.md

Tools used and where they changed decisions.

4. /docs (optional)

Architecture sketch, limitations, LLM pros/cons if used.

Evaluation Focus

We grade, in order:

3. Data + join discipline on this schema
4. Leakage judgment (especially legacy_propensity_score and the region aggregate)
5. Fairness / compliance reasoning (the demographic checkpoint, not just model accuracy)
6. Model evaluation honesty (baselines, subgroup breakdown, calibration if attempted)
7. Agent tool design + both refusal rules
8. Communication / reproducibility

A clean model that refuses leaky shortcuts and reasons carefully about fairness beats a flashy AUC that used them.

Good luck.

We care more about how you think than the last decimal of AUC :)